

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1

Total

Marks:100

Annexure A

Assignment

Code of Conduct:I _____Delisha A K(192511307)_____
(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Delisha

Course Code & Name **CSA11 – Object Oriented Analysis and Design**

Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS)
Course Outcome(s) – CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS). The system shall integrate patients, doctors, reception staff, administrators, laboratory personnel, and pharmacy staff through a centralized intelligent platform. Patients shall register, manage profiles, search doctors, book/reschedule/cancel appointments, receive reminders, view prescriptions and laboratory reports, and submit service requests. Doctors shall manage availability, appointments, clinical notes, prescriptions, and follow-up requests. The system shall provide intelligent doctor/slot recommendations based on specialty, availability, patient preferences, and historical appointment patterns. Laboratory and pharmacy modules shall update report and prescription status. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object responsibilities and apply appropriate design principles and patterns to achieve modularity, maintainability, reusability, security, and scalability.

Constraints

The system shall clearly define the system boundary, actors, objects, classes, attributes, operations, and relationships. UML diagrams shall be consistent with one another, and the operations identified in the Class Diagram shall support the interactions represented in the Sequence and Activity Diagrams.

The design shall demonstrate appropriate use of encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, and

responsibility assignment. Suitable design patterns shall be selected and justified based on the identified design problems.

The final model shall document assumptions and constraints and ensure that all UML models are complete, consistent, unambiguous, traceable, and feasible.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.
Use Case Modelling & Actor	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system	Most actors and use cases are correctly identified with	Basic actors and use cases are identified but

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory
Identification (CO2)		boundary, relationships, and interactions.	minor modelling errors.	relationships system boundary need improvement
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.
UML Architectural Modelling & Design	15	Package, Component, and Deployment Diagrams clearly represent modular structure, system components,	Architectural diagrams are mostly correct with minor omissions or	Basic architectural representation is provided but lacks clarity of

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory
Evaluation (CO3)		dependencies, and deployment environment with strong justification.	modelling errors.	sufficient justification.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application of justification is limited.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated but important links are missing.
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory
				SDG relevance.
Total	100			

Deliverables

To receive full credit, each submission must include the following:

1. **Problem Analysis and Requirement Summary**
2. **Actor Identification**
3. **Use Case Descriptions**
4. **Use Case Diagram**
5. **Object and Class Identification**
6. **Class Diagram**
7. **Sequence/Interaction Diagram(s)**
8. **Activity Diagram**
9. **State Transition Diagram**
10. **Package Diagram**
11. **Component Diagram**
12. **Deployment Diagram**
13. **Object Responsibility Analysis**
14. **Design Axioms/Principles and Design Pattern Justification**
15. **Assumptions and Constraints**
16. **Requirement-to-Use Case/Class Traceability**
17. **Implementation/Prototype and Results**
18. **Reflection on Design Decisions and Learning Outcomes**
19. **GitHub Upload**

1. Problem Analysis & Requirement Summary

The AI-SHAPS system is a centralized platform designed to streamline operations across patients, clinical staff, and administrative personnel.

- **Functional Requirements:**

- **Patient Management:** Registration, profile updates, appointment booking/rescheduling/cancellation, viewing medical history, viewing prescriptions, and viewing lab reports.
- **Doctor Management:** Schedule availability, review patient records, issue clinical notes and prescriptions, and manage follow-ups.
- **AI Recommendation:** Intelligent matching of doctors/time slots based on specialty, past booking patterns, patient preferences, and availability.
- **Lab & Pharmacy Operations:** Laboratory personnel upload/update lab reports; pharmacy staff update prescription fulfillment statuses.
- **System Administration:** User role management, system health monitoring, audit logging, and platform configuration.

- **Non-Functional Requirements:**

- **Security & Privacy:** Encryption of personal health data (HIPAA/GDPR compliance guidelines), role-based access control (RBAC).
- **Scalability & Performance:** Support high concurrent users during peak booking hours; AI recommendation latency $< 200 \text{ ms}$.
- **Maintainability & Modularity:** Decoupled architecture using object-oriented principles and design patterns.

2. Actor Identification Table

Actor Name	Type	Description / Responsibilities
Patient	Primary	Registers, manages profile, books/modifies appointments, views medical records DOCX
Doctor	Primary	Manages schedules, reviews patient records, issues prescriptions & clinical notes DOCX
Reception Staff	Primary	Assists walk-ins, performs manual registrations, manages front-desk scheduling DOCX
Lab Personnel	Primary	Processes tests, uploads diagnostic data, updates lab report status DOCX
Pharmacy Staff	Primary	Accesses prescriptions, dispenses medication, updates fulfillment status DOCX
Administrator	Primary	Manages user roles, system configurations, and views audit logs DOCX
AI Engine	Secondary	Calculates doctor/slot recommendations based on availability & patterns DOCX
Notification Engine	Secondary	Sends automated SMS, email, and push notifications DOCX

3. Use Case Descriptions

Use Case UC-01: Book Appointment with AI Recommendation

- **Primary Actor:** Patient
- **Pre-conditions:** Patient is authenticated and logged into the AI-SHAPS portal.
- **Post-conditions:** Appointment is scheduled, status is set to BOOKED, notification is queued.
- **Main Success Scenario:**
 - Patient selects "Book Appointment" and specifies symptom/specialty.

- System requests recommendations from the AI Recommendation Engine.
- Engine analyzes preferences, doctor schedules, and history, returning ranked slots.
- Patient selects a slot and confirms booking.
- System creates an Appointment record and updates Doctor's schedule.
- System triggers Notification Engine to send confirmation.
- **Alternative Flows:**
 - *3a. AI Engine Unavailable:* System falls back to standard chronological slot filtering by specialty.
 - *4a. Slot Taken:* System alerts patient that slot was just booked and prompts selection of another slot.

4. Object and Class Identification Table

Class Name	Type	Key Attributes	Core Methods / Operations
User	Abstract Class	userID, name, email, phone, passwordHash	login(), logout(), updateProfile()
Patient	Concrete Class	patientID, dob, medicalHistory	bookAppointment(), viewPrescription(), viewLabReport()

Doctor	Concrete Class	doctorID, specialty, qualification	setAvailability(), writePrescription(), addClinicalNote()
Receptionist	Concrete Class	staffID, shiftInfo	registerPatient(), manageBooking()
LabStaff	Concrete Class	labStaffID, department	uploadLabReport(), updateReportStatus()
PharmacyStaff	Concrete Class	pharmacyStaffID	updatePrescriptionStatus(), dispenseMedication()
Appointment	Domain Class	appointmentID, dateTime, status, type	schedule(), reschedule(), cancel()

Prescription	Domain Class	prescriptionID, dosage, instructions, status	createPrescription(), updateStatus()
LabReport	Domain Class	reportID, testType, resultData, status	createReport(), updateStatus()
AIRecommender	System Class	modelVersion, algorithmType	getRecommendedSlots(), analyzePatientPatterns()

5. Requirement-to-Use Case / Class Traceability

Requirement ID	Description	Use Case ID	Classes Involved
REQ-01	Patient registration and appointment scheduling DOCX	UC-01	Patient , Appointment , Doctor
REQ-02	Intelligent slot & doctor recommendation DOCX	UC-01 , UC-02	Patient , AIRecommender , Doctor , Appointment
REQ-03	Clinical notes & prescription issuance DOCX	UC-03	Doctor , Prescription , Patient
REQ-04	Lab test result updates DOCX	UC-04	LabStaff , LabReport , Patient
REQ-05	Pharmacy prescription status fulfillment DOCX	UC-05	PharmacyStaff , Prescription
REQ-06	Automated appointment reminders DOCX	UC-06	Notification , Appointment , Patient

6. Object Responsibility Analysis & Design Principles

Object Responsibilities (GRASP Patterns)

- **Information Controller / Expert:** The Appointment class maintains its state (BOOKED, CANCELLED, COMPLETED) and encapsulates logic regarding valid state changes.
- **Creator:** Doctor creates Prescription instances; LabStaff creates LabReport instances.
- **Low Coupling / High Cohesion:** System logic is divided into explicit packages (e.g., separating AI operations from Core User Management).

Design Axioms & Principles

- **Single Responsibility Principle (SRP):** Notification handles messaging exclusively; it does not process payments or schedules.
- **Open/Closed Principle (OCP):** User is an abstract base class. New roles (e.g., InsuranceAgent) can extend User without modifying existing core login logic.
- **Dependency Inversion Principle (DIP):** AIRecommender communicates with recommendation interfaces rather than hardcoding specific machine learning model logic.

Design Pattern Justification

1. **Factory Method Pattern:** Used for creating different user types (Patient, Doctor, LabStaff) dynamically upon registration.
2. **Observer Pattern:** Used for notification dispatching. When an Appointment status updates, registered observers (NotificationEngine) are triggered to alert the patient.
3. **Strategy Pattern:** Applied within the AIRecommender to switch recommendation algorithms dynamically (e.g., Preference-Based vs. Availability-Based).

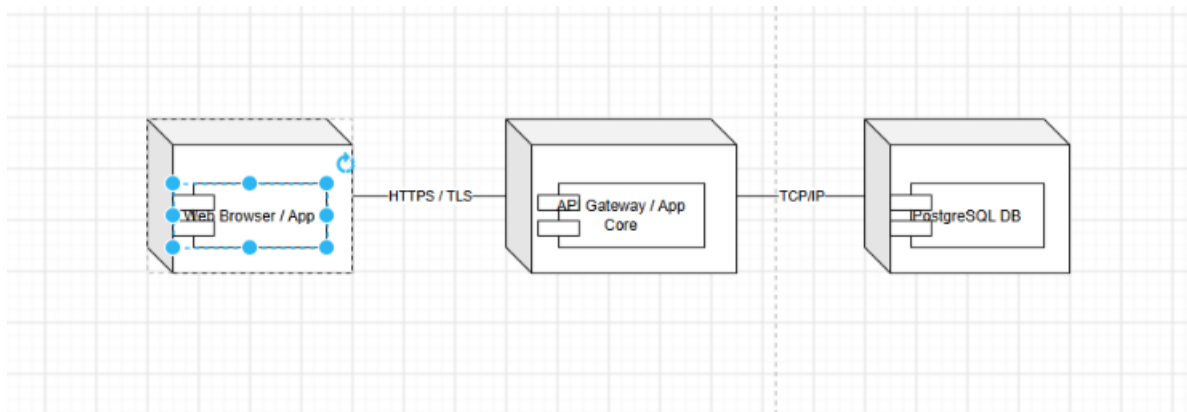
7. Assumptions and Constraints

- **Assumptions:**
 - Users access the system via modern web browsers or native mobile devices.
 - Real-time network connectivity is available for live slot updates.
 - AI model training occurs asynchronously offline; online inference occurs via standard API endpoints.
- **Constraints:**

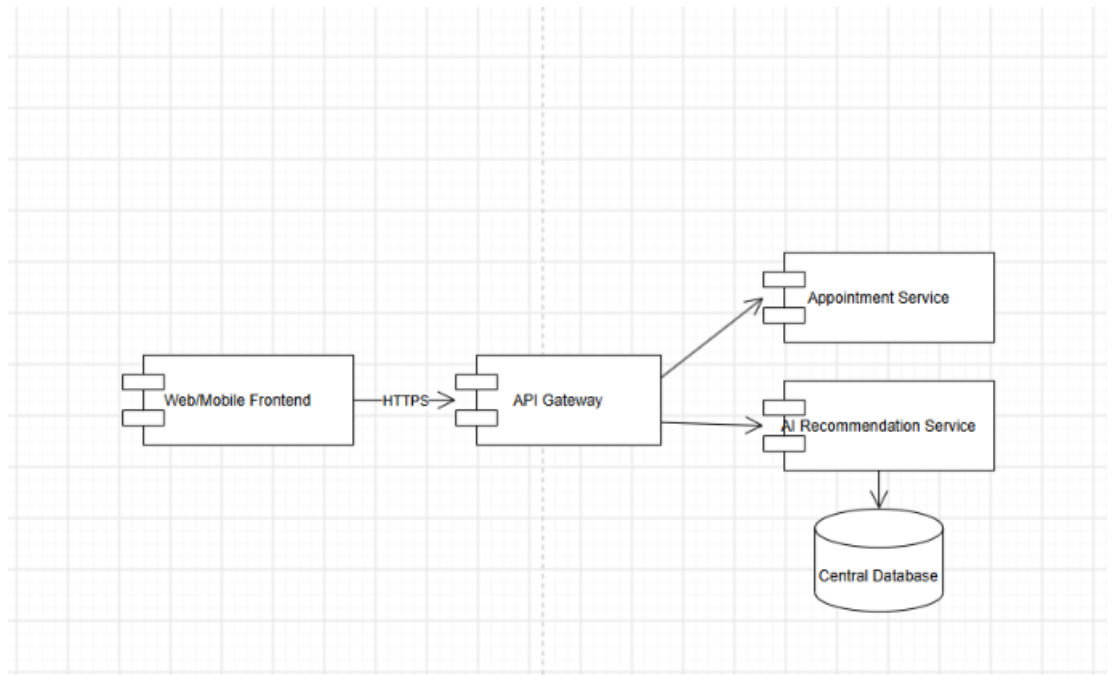
- System must comply with health privacy standards (e.g., HIPAA/GDPR) regarding data isolation.
- Legacy lab equipment integration requires standardized JSON/REST API payloads.

8. PlantUML Diagrams

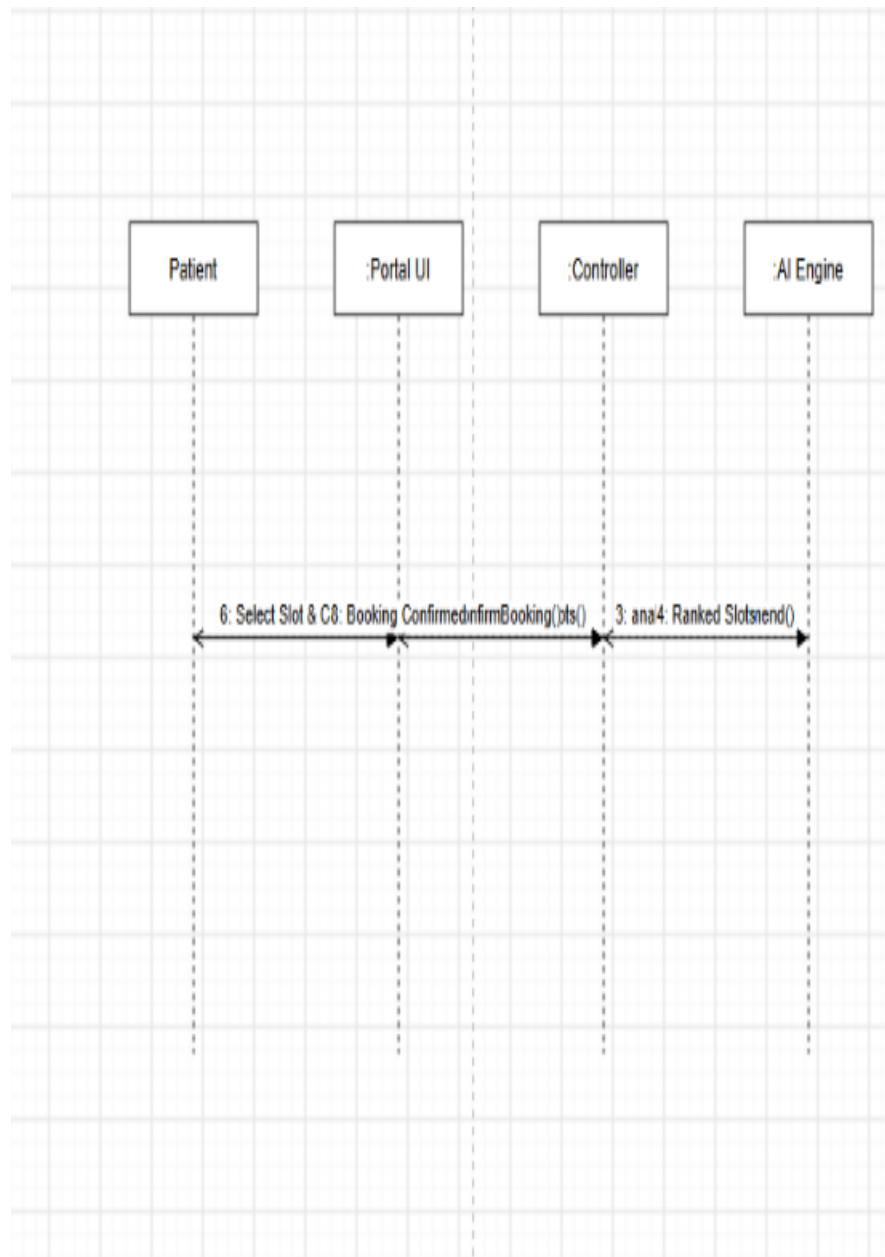
8.1 Deployment Diagram



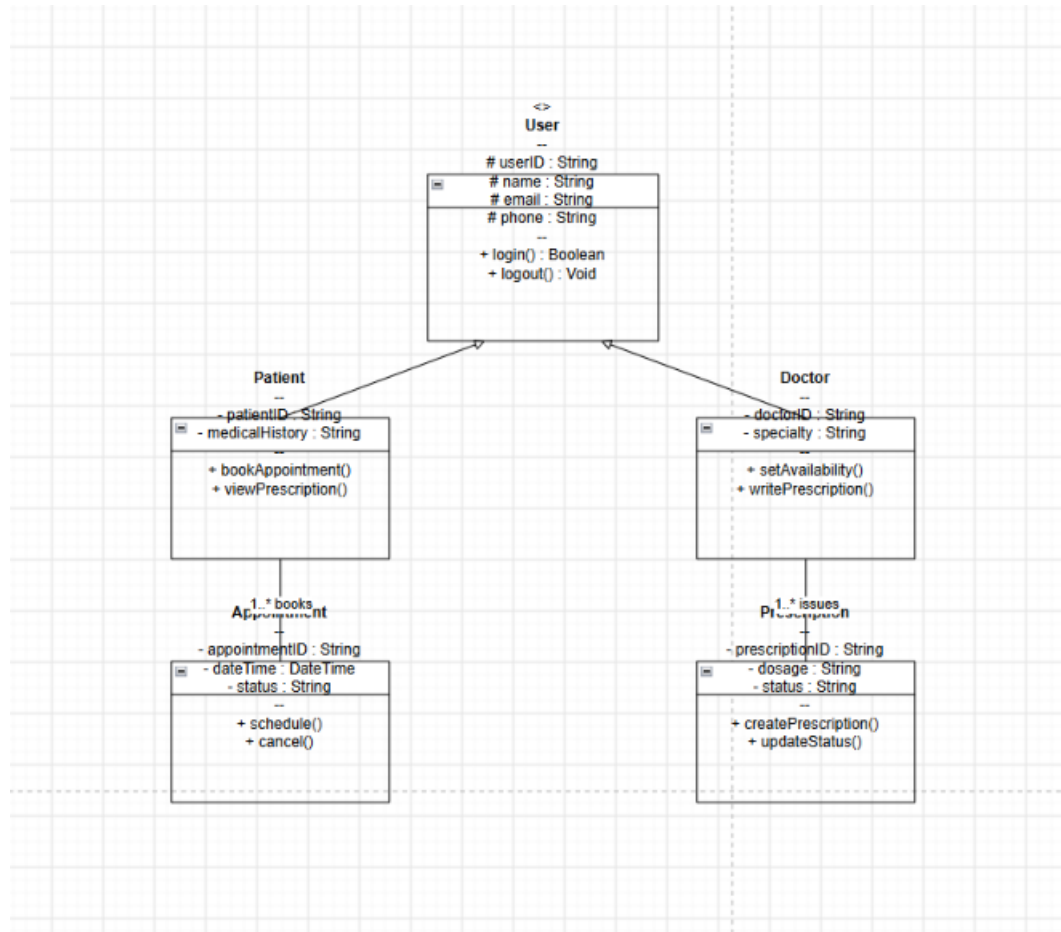
8.2 COMPONENT DIAGRAM



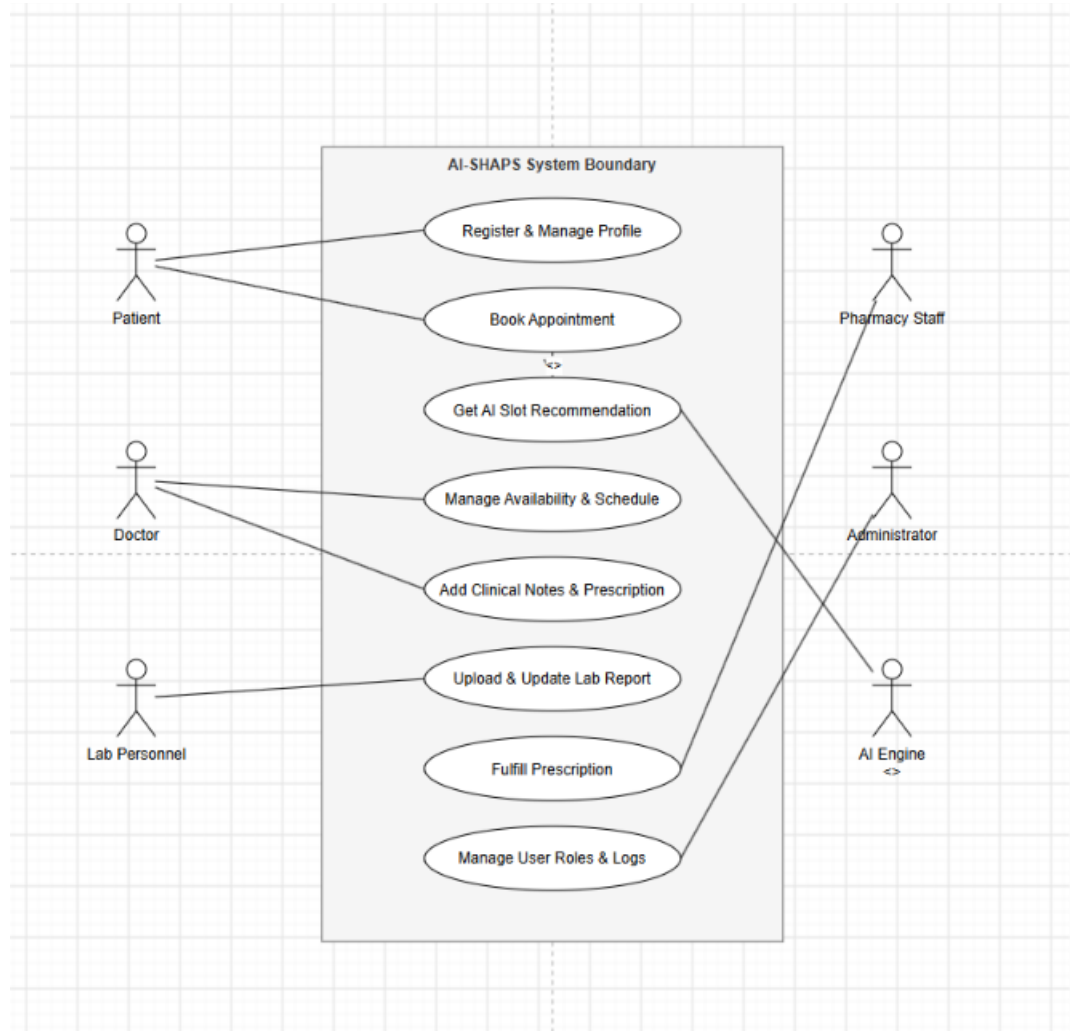
8.3 SEQUENCE DIAGRAM



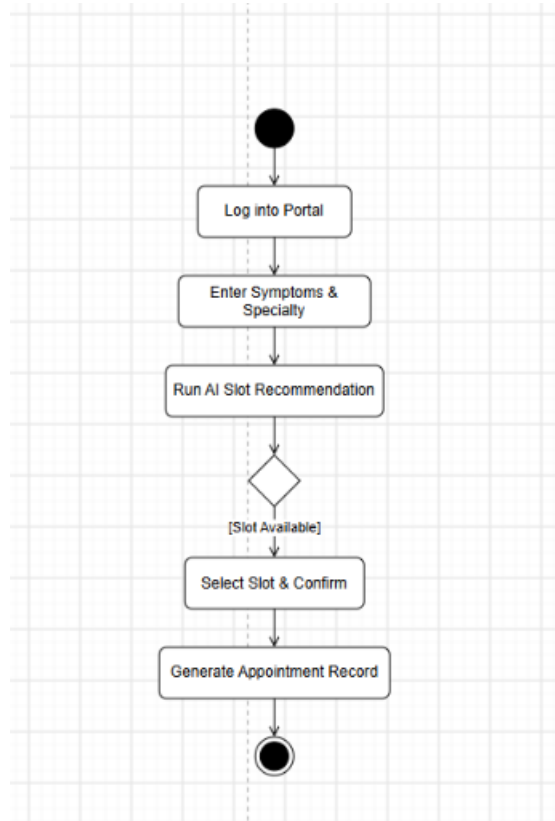
8 CLASS DIAGRAM



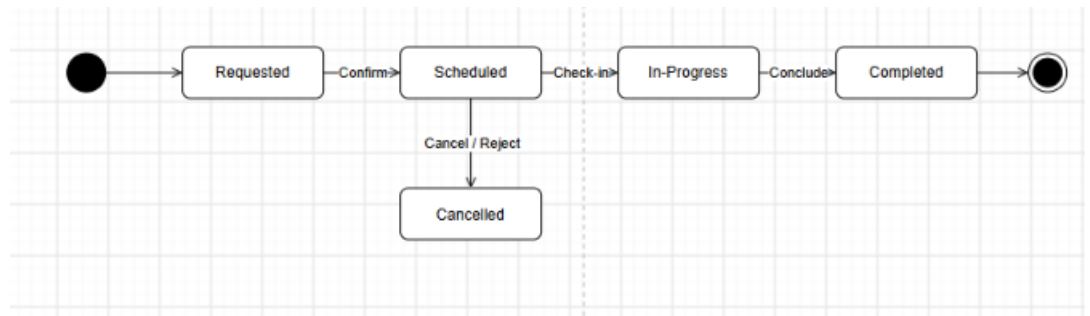
8.5 USE CASE DIAGRAM



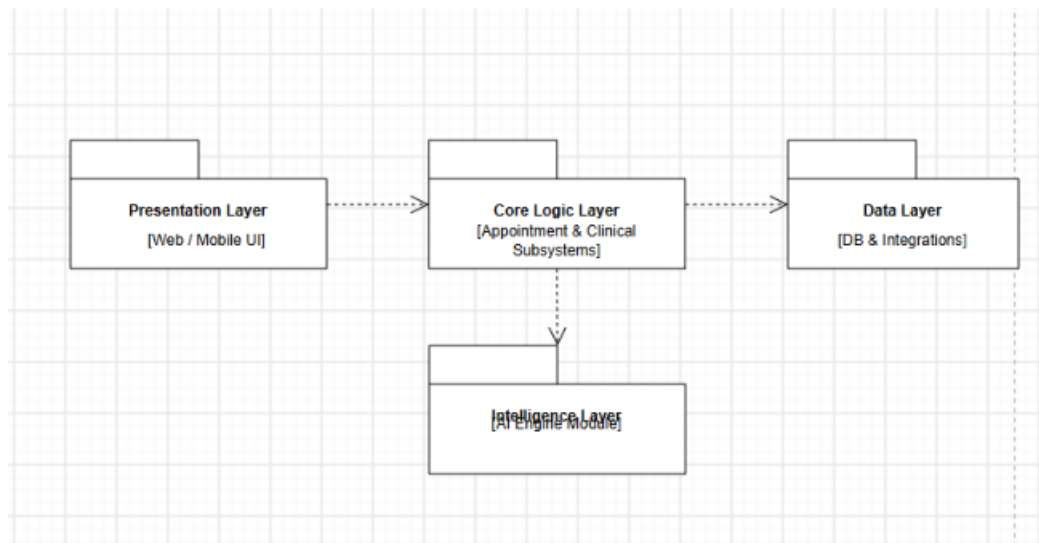
8.6 ACTIVITY DIAGRAM



8.7 STATE TRANSITION DIAGRAM



8.8 PACKAGE DIAGRAM



9. Reflection on Design Decisions & SDG Relevance

Design Reflection & Learning Outcomes

Designing AI-SHAPS emphasized the importance of maintaining traceability across behavioral and structural diagrams. Enforcing GRASP patterns ensured clear class responsibilities and prevented the formation of oversized "God classes." Combining synchronous UI workflows with asynchronous processing (such as notifications and AI model execution) preserves overall system responsiveness and stability.

Relevance to UN Sustainable Development Goals (SDGs)

- **SDG 3: Good Health and Well-being:** Optimizes patient care access, reduces wait times, and improves diagnosis management.

- **SDG 9: Industry, Innovation, and Infrastructure:** Incorporates AI technology and modern cloud infrastructure into healthcare environments.
- **SDG 11: Sustainable Cities and Communities:** Facilitates remote scheduling and resource allocation across urban health networks.